

One Byte, One Rank Reversal: Auditing HumanEval Pipeline Sensitivity for Base Code Models

A Five-Model Study on a Consumer GPU

Jaiveer Bassi

Independent Researcher, USA

jaiveerbassi@yahoo.com

Technical report – primary experiment, diagnostic ablations, and sampling condition

Version 1.9, September 23, 2026

Abstract

A trailing newline appended to stripped HumanEval prompts changed StarCoder2-3B from 3/164 to 49/164 passes in our pinned EvalPlus 0.3.1 pipeline and reversed a five-model ranking. We found this one-byte effect in a post-hoc mechanism audit following a prespecified rank-transfer study; it establishes sufficiency in this pipeline but does not establish the unpublished historical source configuration. The primary study is not an exact re-run because the source’s immutable inference stack and checkpoint revisions are unavailable. Each locally pinned checkpoint produced one greedy BF16 completion for all 164 tasks on an NVIDIA RTX 4070, and a network-isolated container scored HumanEval and HumanEval+. Under the stock prompt boundary, StarCoder2-3B scored 1.8% rather than the displayed 31.7% and reversed order with Qwen2.5-Coder-0.5B. Kendall’s tau-b was 0.8 (task-resampling interval: 0.6 to 0.8), and the paired difference was -21.95 percentage points [-28.66, -15.85], meeting our prespecified failure-to-transfer rule. An unmodified-EvalPlus control scored 1.2% (2/164). Removing only the trailing newline raised StarCoder2-3B to 29.9% (49/164) on HumanEval and 25.6% (42/164) on HumanEval+, while the other four models moved by at most three tasks. The displayed order was then recovered (tau-b = 1.0; interval: 0.8 to 1.0). Benchmark scores for base code models therefore depend on exact prompt boundaries as well as checkpoints.

Keywords: code language models; HumanEval; EvalPlus; reproducibility; local inference; functional correctness

1. Introduction

HumanEval measures whether generated Python functions satisfy hidden tests and remains a common point of comparison for code language models [Chen et al., 2021]. HumanEval+ strengthens that evaluation with additional automatically generated tests [Liu et al., 2023]. Model reports commonly place scores from several checkpoints in one table, inviting readers to interpret the dis-

played order as a portable ranking.

That interpretation can be fragile. Functional-correctness results depend on more than weights: prompt templates, stopping strings, generation length, precision, library behavior, and sanitization can all change the executable completion. These choices matter especially for base models, whose training formats and preferred continuation patterns differ.

We use an external and inspectable target: the ordering of five base-model identifiers in Table 5 of the Qwen2.5-Coder technical report [Hui et al., 2024]. The target contains Qwen2.5-Coder at 0.5B, 1.5B, and 3B parameters, DeepSeek-Coder-1.3B, and StarCoder2-3B. The model families are documented separately by their developers [Hui et al., 2024, Lozhkov et al., 2024, Guo et al., 2024]. Our question is deliberately narrower than exact score replication: does the displayed HumanEval order transfer when locally pinned revisions are run through one disclosed stack on a consumer GPU?

The reusable contribution is a task-level mechanism audit showing that one documented prompt-boundary byte can move one base model by about 28 percentage points while leaving four others nearly unchanged. This post-hoc finding follows a completed five-model primary experiment with a prespecified rank endpoint, paired task-resampling analysis, pinned checkpoint revisions, raw and sanitized completions, full evaluator outputs, and environment metadata. The repository contains the evidence needed to audit both the primary result and the counterfactual [Bassi, 2026].

2. Related Work

HumanEval introduced execution-based functional correctness for 164 handwritten Python synthesis tasks and the pass@*k* family of estimators [Chen et al., 2021]. EvalPlus subsequently expanded the tests associated with HumanEval and showed that test insufficiency can change both absolute scores and model orderings [Liu et al., 2023]. Broader successors expose other limits of

short, static function benchmarks: BigCodeBench emphasizes complex instructions and calls across many libraries [Zhuo et al., 2024], while LiveCodeBench continually adds contest problems to reduce contamination and measure capabilities beyond direct generation [Jain et al., 2024]. Our study does not propose another benchmark. It isolates the transport of one published order across evaluation pipelines while holding model identifiers and functional tests fixed.

Reproducibility also has distinct levels. Sharing code can make one execution repeatable without showing that an empirical conclusion persists across reasonable implementations and environments [Bouthillier et al., 2019]. That distinction motivates our emphasis on prompt bytes, stopping, sanitization, and versioned artifacts: the object being transferred is an ordering produced by an evaluation system, not a property of weights alone.

The Qwen2.5-Coder report compares several open base models under one table, including all five model identifiers studied here [Hui et al., 2024]. StarCoder2 and DeepSeek-Coder are independently documented model families with different training corpora, objectives, and evaluation practices [Lozhkov et al., 2024, Guo et al., 2024]. The EvalPlus leaderboard also lists StarCoder2-3B at 31.7% HumanEval and 27.4% HumanEval+ as greedy direct code completion [EvalPlus Team, 2026a,b]. Those values are identical to Table 9 of the StarCoder2 report [Lozhkov et al., 2024], and the leaderboard row was added on 29 February 2024, before EvalPlus code generation supported StarCoder2. We therefore treat them as one measurement rather than independent replications. We do not construct a new leaderboard or claim contemporary model coverage. We treat the published table as a concrete external rank-transfer target and focus on the otherwise underreported inference boundary between weights and executable benchmark candidates.

3. Research Question and Decision Rule

The primary question is: **Does the HumanEval ordering of the five selected model identifiers in Qwen2.5-Coder Table 5 transfer to the specified local evaluation pipeline?** HumanEval+ is co-reported as a robustness outcome, not substituted for the primary endpoint.

The primary observed statistic is Kendall’s tau-b between the published HumanEval order and the local greedy pass@1 order. We also compare each adjacent pair in the published order using paired resampling over the 164 tasks. Each resample preserves all five model outcomes for a selected task. We use 10,000 percentile-bootstrap replicates and fixed analysis seed 2026. Because all HumanEval tasks are evaluated, these intervals do not quantify uncertainty about the finite benchmark itself; they describe stability to perturbing task composition, or, under an additional exchangeability assumption, uncertainty for a hypothetical population of similar tasks.

The four adjacent intervals are not multiplicity-adjusted. The prespecified decision has three outcomes:

- **Transferred:** the point ordering is identical, with tau-b equal to 1 and no adjacent published pair reversed locally.
- **Failed to transfer:** at least one pair is reversed and the paired 95% task-resampling interval for that pass@1 difference excludes zero in the reversed direction.
- **Inconclusive:** every other outcome, including unresolved ties or reversals whose interval includes zero.

This rule and the primary condition were prespecified in repository commit 1025978 before aggregate correctness results were inspected; we use “prespecified” rather than claiming an independent registry record. The endpoint tests rank transfer, not equality of percentages. Absolute score differences are descriptive because the source paper links model artifacts and evaluation code but does not identify immutable checkpoint revisions or every decoding, stopping, and software detail needed to reconstruct Table 5.

4. Methods

4.1 Models and immutable inputs

Table 1 lists the local base-model revisions. They correspond to the published model identifiers and were resolved and frozen before aggregate correctness results were inspected; the source report does not establish that these are the historical revisions used for Table 5.

4.2 Generation condition

All five models used the EvalPlus base-model HumanEval prompt, with base mode forced even when a tokenizer exposed a chat template. EvalPlus 0.3.1 was selected before aggregate inspection as a fixed, public release supporting HumanEval and HumanEval+; it is an auditable local target, not a claim about the unpublished historical Qwen pipeline. Generation was greedy with one completion per task, a maximum of 512 new tokens, batch size one, and BF16 weights. The models ran sequentially on one NVIDIA GeForce RTX 4070 with 12,282 MiB dedicated memory in WSL Ubuntu on Windows. The generation environment used PyTorch 2.8.0+cu128 and Transformers 4.57.6. Hardware is reported to delimit the stack; no latency, throughput, or cross-hardware conclusion is drawn.

The primary design uses no repeated random seeds. Conditional on the frozen inputs and software stack, greedy decoding is deterministic; repeated identical runs would measure system nondeterminism rather than sampling variance. The separately specified 20-sample sensitivity condition is reported as a secondary result in Section 7; it does not enter the primary endpoint.

Table 1: Frozen model inputs. Full identifiers also appear in machine-readable protocol metadata.

Model	Hugging Face checkpoint	Revision
Qwen2.5-Coder-0.5B	Qwen/Qwen2.5-Coder-0.5B	8123ea2e9354afb7ffcc6c8641d1b2f5ecf18301
StarCoder2-3B	bigcode/starcoder2-3b	733247c55e3f73af49ce8e9c7949bf14af205928
DeepSeek-Coder-1.3B	deepseek-ai/deepseek-coder-1.3b-base	c919139c3a9b4070729c8b2cca4847ab29ca8d94
Qwen2.5-Coder-1.5B	Qwen/Qwen2.5-Coder-1.5B	df3ce67c0e24480f20468b6ef2894622d69eb73b
Qwen2.5-Coder-3B	Qwen/Qwen2.5-Coder-3B	09d9bc5d376b0cfa0100a0694ea7de7232525803

4.3 Functional evaluation and execution safety

The same completions were evaluated on all 164 HumanEval tasks and the corresponding HumanEval+ v0.1.10 tests. We pinned EvalPlus 0.3.1 at commit `e5d0ed0bab96280b60b637ec7f15b5e4841b0cb2`. Generated Python was executed only in a disposable Linux container with networking disabled, credentials absent, capabilities dropped, and no writable host mount beyond the result exchange directory.

The official Docker image tagged v0.3.1 exposed a development package version during verification. We therefore derived the evaluator from the official immutable base-image digest, force-installed released `evalplus==0.3.1`, asserted the installed version at build time, and recorded the derived image identifier.

4.4 Harness equivalence checks

Two changes addressed avoidable generation overhead without changing the declared stopping predicate. First, the upstream batch-one stop criterion decoded the same completion once for each stop string after every token. The wrapper decodes once and checks all stop strings; a pinned one-task comparison required byte-identical raw and sanitized outputs before the full run. Second, EvalPlus left a wrapped Transformers stopping method installed after each task, causing criteria to accumulate. A `finally` block restores the original method after generation. We subsequently repeated HumanEval/0 and HumanEval/1 through unmodified EvalPlus for all five checkpoints. Raw and sanitized records were byte-identical to the retained primary records for every model and task. Machine-readable hashes for both checks are included in the repository.

The provider’s mutable stop list could also acquire duplicate strings when models were constructed sequentially. The runner removes only exact duplicates while preserving first occurrence and order, leaving the logical `any(stop in decoded)` predicate unchanged.

4.5 Analysis

Pass@1 is the proportion of 164 tasks for which the single completion passed. At one completion per task this is a direct proportion, not the multi-sample estimator introduced with HumanEval [Chen et al., 2021]. Task-resampling intervals are computed on paired outcomes and are interpreted as composition-stability analyses, not

uncertainty about the fully observed benchmark. The analysis program rejects missing, duplicate, non-Boolean, or mismatched task records before aggregation.

5. Results

Table 2 reports all primary measurements. Exact counts are included to avoid false precision.

The StarCoder2 discrepancy is not confined to the Qwen comparison table. The EvalPlus leaderboard reports the same 31.7% HumanEval and 27.4% HumanEval+ values for greedy direct completion [EvalPlus Team, 2026a,b]. Those values match the StarCoder2 report’s own Table 9 [Lozhkov et al., 2024], so all three published sources trace to one measurement. Our unmodified-EvalPlus result of 1.2% nevertheless conflicts with that measurement under the same evaluator family and nominal mode, which requires a pipeline-level explanation (Section 6).

The published HumanEval order from low to high was Qwen2.5-Coder-0.5B, StarCoder2-3B, DeepSeek-Coder-1.3B, Qwen2.5-Coder-1.5B, and Qwen2.5-Coder-3B. Locally, StarCoder2-3B moved below Qwen2.5-Coder-0.5B; the other four positions retained their relative order. Kendall’s tau-b was 0.8, with a paired task-resampling 95% interval of [0.6, 0.8]. With only five models, tau-b is coarse; the paired differences below carry more information about the reversal.

For the reversed adjacent pair, the local difference for StarCoder2-3B minus Qwen2.5-Coder-0.5B was -21.95 percentage points, with paired task-resampling interval [-28.66, -15.85]. The interval excludes zero in the reversed direction. The primary decision is therefore **failed to transfer**.

The other adjacent local HumanEval differences, in published low-to-high order, were: DeepSeek-Coder-1.3B minus StarCoder2-3B, +32.32 points [25.61, 39.63]; Qwen2.5-Coder-1.5B minus DeepSeek-Coder-1.3B, +4.88 points [-2.44, 12.20]; and Qwen2.5-Coder-3B minus Qwen2.5-Coder-1.5B, +13.41 points [5.49, 21.34]. These unadjusted intervals are descriptive and do not alter the prespecified decision once a resolved reversal is present.

HumanEval+ preserved the same local order. Qwen2.5-Coder-3B matched the displayed published percentage to rounding on both benchmarks; DeepSeek-Coder-1.3B was 1.9 points higher locally on HumanEval+; and the remaining Qwen checkpoints were 2.5 to 4.9 points lower.

Table 2: Qwen2.5-Coder Table 5, EvalPlus leaderboard, and local primary pass@1. HE denotes HumanEval; HE+ denotes HumanEval+. Dashes indicate models absent from the current EvalPlus result data.

Model	Qwen HE	EvalPlus HE	Local HE	Qwen HE+	EvalPlus HE+	Local HE+
Qwen2.5-Coder-0.5B	28.0%	—	23.8% (39/164)	23.8%	—	20.1% (33/164)
StarCoder2-3B	31.7%	31.7%	1.8% (3/164)	27.4%	27.4%	1.8% (3/164)
DeepSeek-Coder-1.3B	34.8%	28.7%	34.1% (56/164)	26.8%	25.6%	28.7% (47/164)
Qwen2.5-Coder-1.5B	43.9%	—	39.0% (64/164)	36.6%	—	34.1% (56/164)
Qwen2.5-Coder-3B	52.4%	—	52.4% (86/164)	42.7%	—	42.7% (70/164)

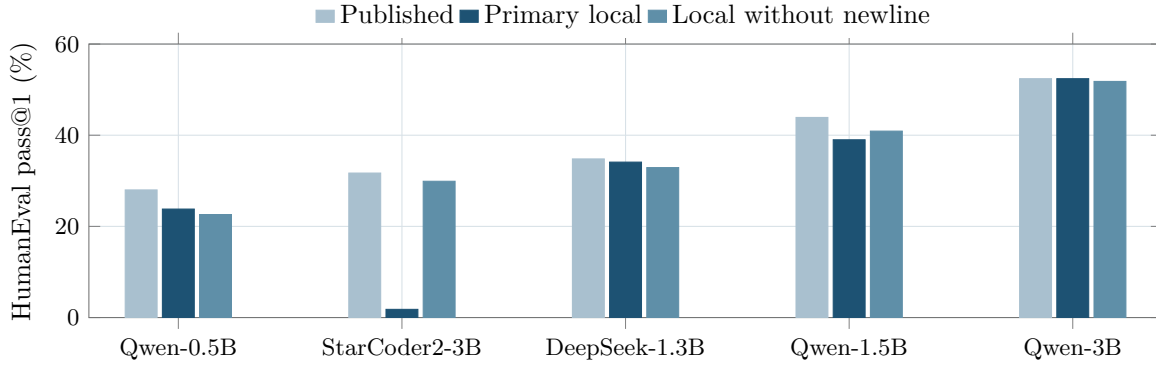


Figure 1: Published and local HumanEval pass@1. Removing the trailing prompt newline is a post-hoc condition; within the pinned local pipeline it recovers the displayed order.

StarCoder2-3B was the clear outlier, 29.9 points below the displayed HumanEval score and 25.6 points below the displayed HumanEval+ score.

6. Prompt-Boundary Mechanism Audit

The StarCoder2-3B result warranted a targeted diagnostic because its absolute difference was much larger than those of the other models. Using the same pinned weights, BF16 precision, and eager Transformers path, the checkpoint generated ordinary Python for the short prompt shown in its model card [BigCode, 2024]. We first checked for a decoding error. EvalPlus’s Hugging Face provider decodes only tokens generated after the input; the retained raw JSONL begins with the HumanEval prompt because the code-generation layer deliberately serializes `prompt + implementation`. The apparent prompt echo in the artifact is therefore not evidence that the provider decoded the full input sequence.

The generated suffix for HumanEval/0 instead begins by repeating the complete function signature and docstring. EvalPlus 0.3.1 registers the exact text `\ndef` as a generation stop for direct HumanEval completion and then truncates at that boundary. Under the standard stop list, the repeated definition is removed and the candidate contains no body. This mechanism is directly visible in the retained stock and ablation records.

We also reran all 164 StarCoder2 tasks through the unmodified EvalPlus 0.3.1 Hugging Face generation path under the same pinned revision, precision, prompt, decoding, attention, and token limit. Hardened evaluation yielded 2/164 (1.2%) on both HumanEval and Hu-

Table 3: StarCoder2-3B primary result, stock control, post-hoc single-factor conditions, and published reference. The published row is not an experimental condition.

Condition	HE	HE+
Primary, standard stops	1.8% (3/164)	1.8% (3/164)
Stock EvalPlus, standard stops	1.2% (2/164)	1.2% (2/164)
Without <code>\ndef</code> stop	10.4% (17/164)	9.1% (15/164)
Without trailing prompt newline	29.9% (49/164)	25.6% (42/164)
Published (StarCoder2 Table 9)	31.7%	27.4%

manEval+, compared with 3/164 (1.8%) for the optimized wrapper. The stock run passed HumanEval/49 and HumanEval/53; the primary run additionally passed HumanEval/50. Thus the wrapper changes do not account for the low StarCoder2 score or the rank reversal. Two-task stock checks for every model were also byte-identical to their retained primary raw and sanitized records.

We then ran a clearly labeled post-hoc secondary ablation that removed only `\ndef` from the stop list. The checkpoint, revision, base prompt, BF16 precision, greedy decoding, batch size, 512-token cap, sanitizer, dataset, and evaluator were unchanged. To avoid the stock provider’s accumulating-hook overhead, the full run used the primary decode-once and hook-restoration implementation; its first 20 raw and sanitized records were byte-identical to a separately retained stock-ablation prefix. Hardened evaluation produced 17/164 (10.4%) on HumanEval and 15/164 (9.1%) on HumanEval+.

Removing the stop recovered 9.15 percentage points on HumanEval and 7.93 percentage points on HumanEval+

relative to the stock control, but gaps of 21.3 and 18.3 points to the published values remained. The stop was where the damage became visible, not why the model restarted the function.

6.1 Trailing prompt newline

EvalPlus 0.3.1 passes `task["prompt"].strip() + "\n"` to direct-completion models. Its history shows that this boundary has changed: a commit on 17 March 2024 titled “fix: starcoder l2r inference” stripped the prompt, and a commit on 3 August 2024 re-appended the newline [EvalPlus Team, 2026b]. Both followed the StarCoder2 report. In StarCoder2’s tokenizer, the newline after a closing docstring is normally merged with the following blank lines and indentation into one token. A prompt ending in an isolated newline token places the model at a boundary it rarely sees inside a function, and at HumanEval/0 it begins a new top-level definition.

A second post-hoc ablation retained every stop text and changed only the model input, omitting that final newline. The stored candidate remained `prompt + completion`, so sanitization and evaluation were unchanged. Because a prompt change could affect every model, we ran it for all five checkpoints and applied the prespecified analysis code descriptively. Table 4 reports the hardened evaluation and paired task transitions.

StarCoder2-3B rose from 2–3 passes to 49/164 (29.9%) on HumanEval and 42/164 (25.6%) on HumanEval+, three tasks below both published values. On HumanEval, 46 StarCoder tasks changed from fail to pass and none changed from pass to fail; on HumanEval+, 40 improved and one regressed. The other checkpoints changed by at most three net tasks, although their paired transitions show offsetting gains and losses. The local order matched the published order exactly: Kendall’s tau-b was 1.0, with a paired task-resampling interval of [0.8, 1.0]. Adjacent differences in published order were +7.32 points [1.83, 12.80] for StarCoder2-3B minus Qwen2.5-Coder-0.5B, +3.05 [-2.44, 8.54] for DeepSeek-Coder-1.3B minus StarCoder2-3B, +7.93 [0.61, 15.24] for Qwen2.5-Coder-1.5B minus DeepSeek-Coder-1.3B, and +10.98 [3.66, 18.29] for Qwen2.5-Coder-3B minus Qwen2.5-Coder-1.5B. Applied descriptively, the prespecified rule would return *transferred*. Because the condition was chosen after the primary result, it does not replace the primary decision. It shows sufficiency within this pipeline, not the historical configuration that produced the published score.

6.2 Newline-by-stop generation factorial

The definition classifier is lexical: it matches an unindented `def` at the start of a line and compares the following name with the task entry point. It does not parse an AST and can match lines inside multiline strings. Thus the reported counts describe definition-like text, not verified executable definitions. The baseline is the optimized primary run; the unmodified-provider control is a sepa-

rate condition.

We subsequently completed the missing fourth generation cell by omitting both the trailing prompt newline and the exact `\ndef` stop, with the same checkpoint, revision, precision, greedy decoder, sanitizer, and 512-token cap. Table 5 classifies all retained raw suffixes as text; generated Python was not imported or executed. Counts under the standard stop are necessarily post-truncation: a matching top-level `def` is removed before serialization.

Removing the newline eliminates empty retained suffixes even with the standard stop. Removing the stop exposes long multi-definition continuations in both prompt conditions, so the stop masks a broad continuation tendency rather than a HumanEval/0-only event. Sanitization makes the functional implication much narrower: 151/164 candidates in the no-newline/no-stop cell are byte-identical to the already evaluated no-newline candidates. The 13 changed candidates were all failures in that baseline, so static comparison predicted bounds of 49–62 HumanEval and 42–55 HumanEval+ passes. Hardened evaluation then scored the fourth cell at 50/164 (30.5%) and 43/164 (26.2%), respectively: one fail-to-pass and no pass-to-fail transition on each suite relative to the no-newline/standard-stop cell. Thus removing the newline accounts for nearly all of the functional recovery; removing the stop mainly reveals otherwise truncated continuation text and changes only one additional task outcome.

7. Stochastic Sensitivity

The prespecified secondary condition draws 20 samples per task at temperature 0.2 with top-p 0.95, seed 11, BF16 weights, and the same prompt, stop texts, 512-token cap, and evaluator as the primary run. It covers the three resource-matched checkpoints named in the plan. We add one post-hoc run: StarCoder2-3B without the trailing prompt newline, so the counterfactual of Section 6 can be checked outside greedy decoding. All four runs completed without interruption, and `pass@k` uses the unbiased estimator [Chen et al., 2021] with paired task-resampling intervals.

Table 6 shows that sampled `pass@1` is numerically within about two points of greedy `pass@1` in every reported condition. Greedy decoding and temperature-0.2 sampling are different estimands, so proximity does not make the greedy output a random draw or establish its typicality. StarCoder2-3B nevertheless remains at 2.3% under the stock prompt across 20 samples per task, and the no-newline condition raises it under both decoding regimes. HumanEval+ `pass@1` follows the same numerical pattern at 31.5%, 27.8%, 2.3%, and 26.1%.

Seeds 23 and 37 were not run. The plan triggers them only on a rank reversal or an adjacent pair below five percentage points; the seed-11 ordering is preserved and the closest adjacent pair differs by 5.37 percentage points. That margin is narrow, so the untriggered condition

Table 4: Passes out of 164 and paired task transitions after removing the trailing prompt newline. F→P and P→F denote fail-to-pass and pass-to-fail transitions.

Model	HumanEval				HumanEval+			
	Primary	No newline	F→P	P→F	Primary	No newline	F→P	P→F
Qwen2.5-Coder-0.5B	39	37	0	2	33	31	0	2
StarCoder2-3B	3	49	46	0	3	42	40	1
DeepSeek-Coder-1.3B	56	54	1	3	47	46	2	3
Qwen2.5-Coder-1.5B	64	67	10	7	56	56	6	6
Qwen2.5-Coder-3B	86	85	6	7	70	71	7	6

Table 5: Text-only classification of 164 StarCoder2 raw suffixes in the post-hoc 2×2 generation factorial. “Repeat” means a top-level definition of the task entry point.

Condition	Empty	Any def	Repeat
Newline, standard stop	142	0	0
Newline, no \ndef stop	2	152	63
No newline, standard stop	0	0	0
No newline, no \ndef stop	0	143	38

Table 6: Twenty-sample HumanEval pass@k (%) with paired task-resampling intervals, beside greedy pass@1 (%).

Condition	pass@1 (%)	pass@5 (%)	Greedy (%)
Qwen2.5-Coder-1.5B	37.6 [31.2, 44.1]	50.8 [43.8, 57.8]	39.0
DeepSeek-Coder-1.3B	32.2 [25.9, 38.7]	41.3 [34.2, 48.6]	34.1
StarCoder2-3B, stock	2.3 [0.9, 4.2]	5.6 [2.8, 8.7]	1.8
StarCoder2-3B, no newline	29.8 [23.7, 36.2]	40.4 [33.4, 47.4]	29.9

should not be read as a wide separation.

DeepSeek-Coder-1.3B generated at batch size 2 rather than 20 because its multi-head key/value cache for 20 concurrent sequences exhausts the 12 GB GPU, after which the driver spills into host memory. Sample count, temperature, top-p, prompt, stop texts, and token cap are identical across conditions. Batch size can affect numerical behavior and random-number consumption; we did not establish sample identity across batch sizes, so this is an additional limitation of cross-model sampling comparisons.

7.1 Additional single-checkpoint sensitivity

The plan assigns two further single-factor conditions to Qwen2.5-Coder-1.5B at seed 11, holding the reference sampling settings fixed. The first stops forcing the base prompt: the checkpoint ships a chat template, so EvalPlus builds its instruction-style prompt using its own default instruction and response prefixes. The second loads bitsandbytes 8-bit weights in place of BF16. Each run verifies at load time that the intended path was taken, so a silent fallback to the reference condition fails rather than producing a misleading number. Differences are paired: the 164 tasks are resampled with each task’s two per-task pass@1 estimates kept together.

Table 7: Qwen2.5-Coder-1.5B single-factor sensitivity conditions against the 20-sample reference, with paired task-resampling intervals on the pass@1 difference.

Condition	pass@1 / pass@5 (%)	Δ pass@1 [interval] (points)
Reference (BF16, base prompt)	37.6 / 50.8	—
Chat-template prompt	48.6 / 66.5	+11.07 [4.39, 17.74]
8-bit weights	24.4 / 34.4	−13.17 [−17.90, −8.66]

Table 7 shows that both intervals exclude zero. HumanEval+ moves in the same directions, to 43.7% for the chat prompt (+12.23 [5.64, 18.96]) and 23.2% for 8-bit (−8.23 [−12.29, −4.24]). These conditions cover one checkpoint at one seed and do not bear on the primary decision. They show that these two choices materially affect Qwen2.5-Coder-1.5B in this stack; they do not bound prompt or quantization effects for other checkpoints or pipelines.

8. Threats to Validity

Target specification. The published percentages and model identifiers are available in Qwen2.5-Coder Table 5, but the report does not specify every generation and post-processing detail or immutable checkpoint revision for that table. The current public evaluation-code link associated with the report is not, by itself, an immutable reconstruction of the historical run. Our pinned revisions therefore correspond to the named artifacts but are not proven to be the historical snapshots. The experiment tests transfer to a disclosed condition, not a faithful rerun of an undisclosed condition.

Single deployment stack. The primary run used one GPU, one precision, one inference library version, and one evaluator release, and does not estimate variation across hardware, kernels, or libraries; it also does not support CPU performance claims. The prompt and 8-bit ablations of Section 7 probe two of those axes on a single checkpoint and are not a general characterization of prompt mode or quantization.

One greedy completion. The primary endpoint measures deterministic greedy pass@1. The secondary 20-sample condition in Section 7 characterizes sampling at one temperature and one seed for three checkpoints; it does not cover the remaining models, other temperatures,

or repeated seeds. Greedy and sampled pass@1 are different estimands. The task bootstrap perturbs task composition, not sampler seeds.

Statistical scope. HumanEval is fully enumerated here. Task-resampling intervals require an exchangeability interpretation for inference beyond these 164 tasks and otherwise serve as stability summaries. Kendall’s tau-b is discrete with five models, and the four adjacent intervals are not multiplicity-adjusted. The large StarCoder2 reversal is not close to zero, but the intervals should not be read as simultaneous family-wise coverage.

Benchmark scope. HumanEval contains 164 Python tasks. HumanEval+ adds tests but not independent tasks, so it does not increase the task-level sample size. We did not audit benchmark contamination, language generality, security, maintainability, or real repository-level coding ability.

Multiplicity sensitivity. A post-hoc check of the primary reversed pair finds 37 tasks passed only by Qwen-0.5B and one only by StarCoder2. The two-sided exact McNemar probability is 2.84×10^{-10} , or 1.14×10^{-9} after Bonferroni adjustment for the four adjacent pairs. This additional analysis leaves the prespecified decision intact. It assumes a paired-task inference model and does not establish representative sampling of real programming tasks. Counts and calculations are reproducible with `scripts/audit_paper_evidence.py`.

Diagnostic scope. The StarCoder2 probe and ablations were selected after observing the primary outcome and are explanatory rather than confirmatory. The prompt-newline condition closes all but three tasks of the StarCoder2 gap, but the code that produced the published values is not public, so we cannot show that it used the same prompt boundary; the small residual may reflect other library or tokenizer differences. The retained generations and hardened scores complete the 2×2 newline-by-stop design across all 164 tasks. No pre-truncation logits were retained, so the factorial establishes a local behavioral interaction without identifying the historical source configuration or fully resolving the tokenizer-level mechanism.

Secondary-condition scope. The chat-prompt and 8-bit sensitivity conditions cover only Qwen2.5-Coder-1.5B at one seed. They demonstrate effects for that checkpoint and stack, not general prompt-mode or quantization effects.

Temporal scope. We study transfer of a 2024 comparison using locally frozen revisions corresponding to its named models. This is not a claim that the five models represent the strongest code models available in 2026.

9. Reproducibility and Artifact Availability

The public repository [Bassi, 2026] contains:

- the prespecified question, endpoint, decision rule, and conditional secondary plan from commit 1025978;
- five immutable checkpoint revisions and the published target table in JSON;
- raw and sanitized completions for all 820 model-task pairs;
- five complete evaluator result files and 820 Boolean task outcomes;
- the generated analysis JSON, runtime package freeze, GPU metadata, Git state, evaluator image identifier, and SHA-256 checksums;
- the generation, validation, consolidation, analysis, and hardened evaluation code;
- equivalence records for the generation-wrapper changes, including two-task stock controls for all five models;
- the full stock StarCoder2 generation, hardened evaluator output, image identifier, and task-level comparison;
- the complete newline-def stop ablation, 20-task stock-path equivalence prefix, hardened evaluator output, run identifier, and checksums;
- the five-model trailing-newline ablation, including generations, hardened evaluator outputs, 820 task outcomes, analysis JSON, and checksums;
- the fourth newline-by-stop cell, all-task text classification, sanitized-candidate comparison, hardened evaluator output, image identifier, and workflow run;
- the four 20-sample sensitivity runs, with 13,120 completions, hardened evaluator outputs, pass@ k analysis, and checksums; and
- the Qwen2.5-Coder-1.5B prompt and 8-bit ablations, with generations, hardened evaluator outputs, paired analysis, and checksums.

The artifact directory contains model-generated Python and should be treated as untrusted data. Re-evaluation should use the documented disposable container rather than executing samples on a host system. The matching software, artifact, and manuscript release v1.9.0 is archived under DOI [10.5281/zenodo.22922348](https://doi.org/10.5281/zenodo.22922348); the concept DOI [10.5281/zenodo.22800650](https://doi.org/10.5281/zenodo.22800650) resolves to the latest release. The earlier v1.3.0 archive remains available under DOI [10.5281/zenodo.22848609](https://doi.org/10.5281/zenodo.22848609).

Release boundary. The v1.9.0 DOI archives this manuscript and its matching evidence, including the completed factorial and subsequent review fixes. The v1.3.0 DOI identifies the earlier software/artifact release and does not contain those later additions.

10. Conclusion

The published five-model HumanEval ordering did not transfer to our specified local pipeline. A single resolved reversal reduced Kendall’s tau-b to 0.8 (task-resampling interval: 0.6 to 0.8) and met the prespecified failure-to-transfer rule. The result was driven by StarCoder2-3B, and its 1.2% unmodified-EvalPlus control shows that the optimized wrapper is not sufficient to explain the low score. In a post-hoc counterfactual, removing the trailing newline EvalPlus 0.3.1 appends to base-model prompts raised StarCoder2-3B to 29.9%, three tasks below its published value, and recovered the displayed order.

The practical lesson is not that published tables are unusable. It is that their order should not be treated as checkpoint metadata. Within this pinned stack, one prompt-boundary choice left four models nearly unchanged and moved a fifth from about 2% to about 30%. That establishes a large local pipeline effect, not the unpublished historical cause of the source score. For base code models, benchmark results are properties of complete evaluation systems. Reports should preserve exact prompt bytes, decoding, stopping, sanitization, evaluator version, checkpoint revision, and task-level outputs; prompt-construction changes should be checked for tokenizer-boundary effects across model families.

References

- Jaiveer Bassi. One byte, one rank reversal: Auditing HumanEval pipeline sensitivity for base code models. Software and research artifacts, version 1.9.0. Zenodo, doi:10.5281/zenodo.22922348, 2026. URL <https://github.com/imjbassi/local-deployment-of-transformer-based-code-assistants>.
- BigCode. StarCoder2-3B model card. Hugging Face, 2024. URL <https://huggingface.co/bigcode/starcoder2-3b>. Accessed 2026-09-14.
- Xavier Bouthillier, César Laurent, and Pascal Vincent. Unreproducible research is reproducible. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 725–734, 2019. URL <https://proceedings.mlr.press/v97/bouthillier19a.html>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- EvalPlus Team. EvalPlus leaderboard and result data. Project leaderboard and machine-readable results, 2026a. URL <https://evalplus.github.io/leaderboard.html>. Machine-readable result data in the project repository; accessed 2026-09-16.
- EvalPlus Team. EvalPlus: Rigorous evaluation of language models for code. Software documentation, 2026b. URL <https://github.com/evalplus/evalplus>. Accessed 2026-09-16.
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024. URL <https://arxiv.org/abs/2401.14196>.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. URL <https://arxiv.org/abs/2409.12186>.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2305.01210>.
- Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024. URL <https://arxiv.org/abs/2402.19173>.
- Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024. URL <https://arxiv.org/abs/2406.15877>.